

Author Attribution using Word2Vec

NLP Assignment 1 Report

Laksh Goel

Preprocessing

I found that the training data contained catalog noise (prices, addresses) mixed into author text, which I removed with targeted regex before tokenisation. I use a punctuation-retaining tokeniser since patterns like semicolons and dashes are strong stylometric signals. I also collapse repeated punctuation, replace digits with <NUM>, and normalise Unicode variants of quotes, dashes, and ellipses to their ASCII equivalents. The vocabulary includes only tokens with frequency ≥ 5 , with special tokens <UNK> and <PAD>.

Word2Vec Model

I train a **Skip-gram with Negative Sampling (SGNS)** model from scratch in PyTorch. The loss for each (target, context) pair is:

$$\mathcal{L} = -\log \sigma(\mathbf{w}_t^{in} \cdot \mathbf{w}_c^{out}) - \sum_{k=1}^K \log \sigma(-\mathbf{w}_t^{in} \cdot \mathbf{w}_{neg_k}^{out})$$

where $K=5$ negatives are sampled from $P(w) \propto f(w)^{0.75}$, following the original paper. I train with Adam ($lr=0.001$), window size 5, batch size 512, for up to 40 epochs.

Inference Representation

I embed each text by first removing a set of high-frequency grammatical stopwords (*the, and, was, he, she, etc.*) that carry little stylometric signal, then mean-pooling the *averaged* input and output embeddings of the remaining tokens:

$$\mathbf{v}_{text} = \frac{1}{|T|} \sum_{t \in T} \frac{1}{2} (\mathbf{w}_t^{in} + \mathbf{w}_t^{out})$$

Both stopword removal and using $(\mathbf{W}_{in} + \mathbf{W}_{out})/2$ instead of $\mathbf{W}_{in}$ alone were motivated by error analysis on 12 hard queries — this averaging produces more isotropic embeddings when training data is limited.

Task 1

I embed the query and each candidate, then rank candidates by cosine similarity $\text{sim}(q, c) = \frac{\mathbf{v}_q \cdot \mathbf{v}_c}{\|\mathbf{v}_q\| \|\mathbf{v}_c\|}$.

Task 2

Since test chunks come from unseen authors, I first scan them for OOV words and expand $\mathbf{W}_{in}$, $\mathbf{W}_{out}$ with randomly initialised rows while preserving all pre-trained weights. I then fine-tune the full model on the test chunks (up to 200 epochs / 25 min) to adapt embeddings to the test distribution. Finally, I cluster L2-normalised chunk embeddings using `KMeansConstrained`, which enforces the `min_chunks_per_author` constraint that standard K-Means ignores.

Experiments (What Did Not Help)

CBOW. I implemented Continuous Bag-of-Words as an alternative to Skip-gram. CBOW averages context embeddings to predict the target word, which tends to smooth out rare word representations. It performed worse than Skip-gram on author ranking, likely because rare, author-specific vocabulary is precisely what matters for stylometry.

Distance-weighted context windows. Motivated by the Word-Space Model, I weighted context words by their distance from the target: GloVe-style ($1/d$) and an aggressive exponential scheme ($1/2^{d-1}$). The intuition was that closer context words carry stronger stylistic signal. In practice, neither scheme improved over uniform weighting, suggesting the window-5 context already captures the relevant neighbourhood adequately.

Doc2Vec / Paragraph Vectors. I added a document embedding matrix $\mathbf{D}$ that concatenated an author-level vector into the SGNS loss, so each author’s style could be represented as a trainable vector alongside word embeddings. This did not generalise to unseen authors at test time, which is exactly the setting that matters most for this task.

Smooth Inverse Frequency (SIF). Instead of uniform mean-pooling, I weighted each token’s embedding by $\frac{a}{a+p(w)}$ (down-weighting frequent words) before averaging. While theoretically appealing, SIF did not outperform plain mean-pooling with stopword removal, suggesting that the explicit stopword filter already handles the high-frequency noise problem sufficiently.

Subsampling. I experimented with probabilistic subsampling of frequent tokens during training ($P(\text{keep}) = \sqrt{t/f(w)}$, as in the original Word2Vec paper). This had negligible effect, again pointing to small corpus size as the bottleneck rather than frequency imbalance.

Stopword Removal Strategy

I experimented with stopword removal at two different stages of the pipeline, and found that *where* you remove them matters significantly.

During training (did not help). I first tried excluding stopwords from the training corpus entirely, so the Skip-gram model would never see them as target or context words. The training stopword set was: **the, a, an, and, or, but, in, on, at, to, for, of, is, was, are, were, been, be, have, has, had, do, does, did, will, would, should, could, may, might, must, can, i, you, he, she, it, we, they, this, that, these, those, s, t.** This hurt embedding quality — these words still participate in syntactic co-occurrence patterns that help anchor the geometry of the embedding space, and removing them produced worse author rankings.

At inference time (helped significantly). I instead kept the full vocabulary during training and applied a stopword filter only when aggregating token embeddings into a text representation. The inference stopword set was: **,, ., the, and, of, to, a, in, i, was, that, it, ", -, ', he, you, she, his, had, her, with, for, as, at, is, ;, on, but, s.** By excluding these before mean-pooling, the final text vector reflects stylistically meaningful content words rather than grammar. This separation — *train on everything, filter only at aggregation* — gave a significant improvement in author ranking accuracy.

Design Decisions

Component	Choice
Architecture	Skip-gram with Negative Sampling
Embedding dim / Window / Negatives	100 / 5 / 5
Noise distribution exponent	0.75
Optimizer	Adam, $lr = 0.001$
Min vocab frequency	5
Inference representation	Mean of $(\mathbf{W}_{\text{in}} + \mathbf{W}_{\text{out}})/2$, stopwords removed
Task 1 similarity	Cosine similarity
Task 2 clustering	Constrained K-Means on L2-normalised embeddings